Akka vs. PydanticAI

A comparison for teams building agentic AI

October 2026

PydanticAI gives you a clean, type-safe way to build agents in Python; Akka gives you the Akka Agentic AI Platform, which runs them and guarantees resilience and scalability. PydanticAI is a typed Python agent framework — model-agnostic, with structured outputs, dependency injection, and graph support. It is a framework, not a platform: no durable runtime of its own, no HA/DR, no operational SLA, and guardrails that cover PydanticAI agents only. You assemble and operate production yourself.

0	99.9999%	8	90%
PYDANTICAI FRAMEWORK SLA	AKKA AGENTIC AI PLATFORM SLA	EXTERNAL ENGINES PYDANTICAI LEANS ON	CHEAPER TO OPERATE ON AKKA

DIMENSION	PYDANTICAI	AKKA
What it is	A typed Python agent framework (library)	The Akka Agentic AI Platform, which guarantees resilience and scalability
Scope	Agent construction, structured output, tools, MCP, graph; durability, HA/DR, governance, and operations are the customer's	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
Durable execution	None native — wrapped around one of eight external engines (Temporal, DBOS, Prefect, Restate, AWS Lambda, and others)	Built into the runtime; event-sourced, replayable
Availability SLA	None on the framework; SLA only on Logfire Enterprise observability	99.9999% — entire platform, backed by indemnities
RTO / RPO	Owned by the customer's chosen infrastructure	Sub-1-minute RTO; zero-byte RPO
Memory / state	Process memory; Harness step persistence to a store you choose; crash recovery via an external engine	Durable in-memory, 4ms reads / sub-10ms writes
Governance / EU AI Act	Inline guardrails and human tool approval in agent code (Harness); Logfire tracing and evals; no named AI regulation	Aspect-woven runtime enforcement + full pre-production governance
Real-time streaming	Not provided; sourced separately	Built in, backpressured, petabyte-scale, no external broker
Cost model	Python infrastructure footprint you provision and operate	Shared compute; up to 90% lower infrastructure for the same workload
Certifications	SOC 2 Type II, HIPAA, GDPR (Logfire)	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF)

PydanticAI Is a Framework; Akka Is a Platform

PydanticAI is a typed Python agent framework from the team behind Pydantic. Its design point is developer experience: model-agnostic agents, structured and validated outputs, dependency injection, MCP support, and graph-based multi-agent workflows, all checked by your type checker and IDE. For teams building agents in Python who want errors caught at write-time, that DX is genuinely clean. A framework defines how you write code; a platform runs it and guarantees it. The durable runtime, high availability, disaster

recovery, streaming, the API tier, governance beyond each agent's own guardrails, and the operational SLA are **not in the framework** — you source, integrate, and operate them, and own every failure across the seams.

Everything an agentic system needs runs in one Akka runtime, so a system that compiles is ready to run in production. The Akka SDK models the entire agentic system in one place — agents, tools, durable memory, streaming, APIs, and governance — and the platform generates it, tests it, and runs it as one certified service.

Capability	PydanticAI	Akka
Typed agent construction (tools, structured output, MCP, graph)	Yes — clean, type-safe	Yes
Durable runtime	Delegated to external engines	Built in
High availability / disaster recovery	Customer-owned	Built in, active-active
Durable memory	Customer-owned (Harness store or engine DB)	Built in, 4ms / sub-10ms
Real-time streaming	Not provided	Built in, backpressured, petabyte-scale
HTTP / gRPC API layer	Not provided	Built in
Governance / policy enforcement	Inline guardrails and tool approval in agent code; Logfire evals	Inline, runtime-embedded
Operational SLA	None on the framework	99.9999%, entire platform

Reliability, SLA, and Durability

PydanticAI publishes no availability SLA, because a framework does not run anything — availability is a property of the infrastructure the customer builds around it. For durability, PydanticAI leans on external durable-execution engines: it supports eight engines, **Temporal, DBOS, Prefect, Restate, AWS Lambda, Absurd, pgtask, and Apache Airflow**, each of which checkpoints or replays state in its own datastore. The framework documents this explicitly; durability is delegated, not native. Reliability, recovery, and the SLA for the running system are owned and operated by the customer and their chosen engines. Akka delivers durability and availability as runtime properties: state is event-sourced and replayable from the journal, recovery is automatic, and the platform carries a contractual six-nines SLA.

Metric	PydanticAI	Akka
Availability SLA	None on the framework; SLA only on Logfire Enterprise	99.9999% — entire platform
Allowed downtime / year	Whatever the assembled stack delivers	~31 seconds
RTO	Owned by the customer's chosen engine	Sub-1 minute
RPO	Owned by the customer's chosen engine	Zero byte
Durable execution	Delegated to one of eight engines (Temporal, DBOS, Prefect, Restate, others)	Native, in the runtime
SLA scope	None	The entire platform

Up to 90% Cheaper to Operate

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required for the same agentic transaction volume, not list price. A PydanticAI deployment carries the Python infrastructure footprint, plus the separate datastore and operations of whichever durable-execution engine provides recovery, plus the memory, streaming, API, observability, and governance tiers run alongside it.

Akka runs all of it on one shared-compute runtime. The efficiency comes from actor concurrency (~10T tokens/core/year vs ~2T; ~80% less compute than Python frameworks; Manulife: up to 300% more concurrency, 30–50% faster processing after porting from Python), shared compute, and micro-checkpointing. The spend is predictable — a fixed annual fee, not a footprint that scales with load.

Maturity and Vendor

PydanticAI reached **V1 in September 2025** and V2 on June 23, 2026; the current release is 2.61.0 (October 2026), and the 1.x line receives security fixes only. The framework is young but real. Pydantic the company is venture-funded — a **$12.5M Series A led by Sequoia** (Oct 2024), ~$17.2M total — and monetizes through Logfire.

Akka is profitable and self-funding, operating since 2009 across 100,000+ production deployments, and 52 banks behind it. Dell Technologies Capital is the largest shareholder, a customer, and an AI partner — so the platform you standardize on does not depend on a venture timeline.

Continuous AI Intelligence

Akka Optimize keeps improving the AI after it ships. It runs an evaluation loop on your own production traffic, grading live interactions and training smaller specialized models on your proprietary data, so the system keeps getting better as it runs. The loop delivers three results that compound over time:

- **Model leverage** — route each request to the best model from any vendor under your policies, and reserve frontier models for the requests that need them.
- **Data sovereignty** — the specialized models are trained on your proprietary data, owned by you, and run inside your own environment.
- **Cost governance** — full visibility and control of AI spend, with savings that compound as the loop keeps running.

The evaluations run continuously inside Akka and grade traffic from agents wherever they run, in Akka or in third-party harnesses.

Governance and the EU AI Act

PydanticAI enforces guardrails inside each agent. Pydantic AI Harness guardrails block or redact a prompt before the model call, block a tool call before it runs, and hold a tool call for human approval. Engineers write each guardrail into the agent's Python code, and the guardrails cover PydanticAI agents only. Pydantic Logfire, the team's commercial product, traces runs and scores a sample of live traffic with online evaluations, with SOC 2 Type II, HIPAA, and GDPR for the observability service and an EU data region. Logfire keeps traces for 30 days on Team and up to 90 days on Growth. Pydantic maps its controls to no AI regulation such as the EU AI Act.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover

High-risk AI carries a 10-year logging-retention obligation (Art. 72).

How Akka governs

At the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; classification against 241 regulations and frameworks and 1,718 controls (699 carrying a financial penalty) before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Governance a PydanticAI team writes into each agent, Akka Verify applies to any AI system.

Two Lifecycles, One Certified System

Building with PydanticAI means engineers writing Python agent code, with Agent Specs that move an agent's model, instructions, and capabilities into a YAML or JSON file. PydanticAI has no built-in governance lifecycle for a risk officer to author. Akka runs two independent lifecycles on one platform via **Akka Specify**:

BUILD LIFECYCLE

FUNCTIONAL CONTRACT

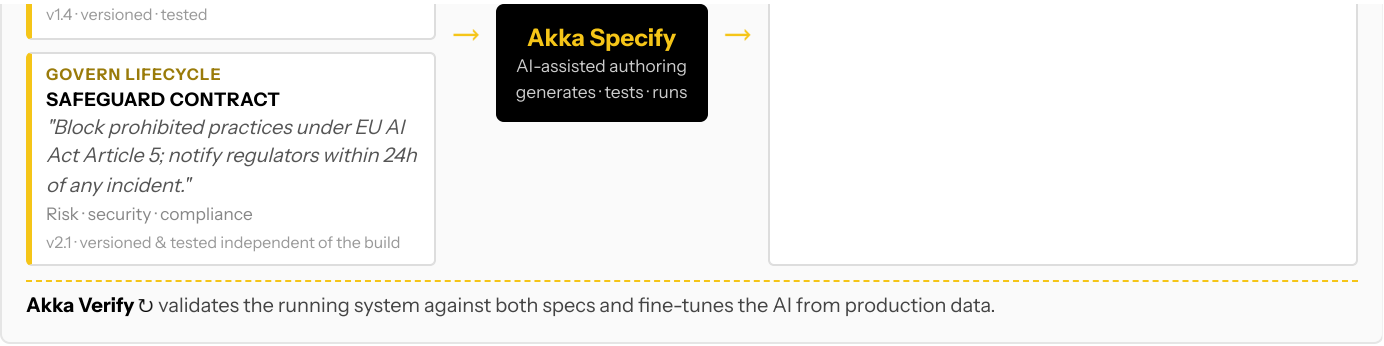
"Rank incoming ER patients by acuity and route the top three to a clinician."

Product · developers · ML engineers · domain experts

ONE CERTIFIED AI SERVICE

Built, governed, and running

- Agents, tools, orchestration, memory, APIs, streaming, UI
- Guardrails, sanitizers, HITL/HOTL, evaluations, halts
- Interaction, evidence, and causal logging



The build lifecycle and the governance lifecycle are versioned and tested independently, by different audiences — an audience and a workflow PydanticAI has no equivalent for.

Real-Time Streaming at Petabyte Scale

PydanticAI has no streaming engine; real-time pipelines are provisioned separately. Akka streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

For the Buyer: Risk, Compliance, and Accountability

Buyer concern	PydanticAI	Akka
Certifications & audits	SOC 2 Type II, HIPAA, GDPR (Logfire observability)	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Scope of accountability	The framework; you integrate and operate the runtime, durability, and everything else	One platform, one SLA, 24/7 SRE — Akka owns the running system
Risk transfer	Standard open-source / cloud terms	Availability and data-integrity guarantees backed by contractual indemnities
Track record & funding model	Venture-funded: $12.5M Series A led by Sequoia (Oct 2024), ~$17.2M total; monetizes via Logfire	Profitable and self-funding; since 2009 and 100,000+ deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Python infrastructure footprint you size and operate	Fixed annual fee finance can forecast

PydanticAI gives you a clean, type-safe way to author agents in Python. Akka gives you the platform that runs them: durable runtime, high availability, embedded governance, and an SLA Akka owns. The decision is scope and accountability — a framework you build a production system around, versus the production system.

Customers Running Agentic and Real-Time Systems on Akka

Manulife
2,000
developers across 100 projects on one governed platform

Tubi
5B tok/s
real-time hyper-personalization engine

Swiggy
71ms
order-assignment AI, ~50% latency reduction

John Deere
1,000+
tractor sensors turned into real-time insight

Verizon
750%
order-processing capacity gain; 6s → 2.4s response

Common Questions

We like PydanticAI's typed developer experience. Why add Akka?

PydanticAI's type-safe DX is a real strength for authoring agents in Python. It is a framework, so the durable runtime, high availability, disaster recovery, streaming, the API tier, embedded governance, and the operational SLA are yours to build and run. Akka delivers those as one platform with a 99.9999% SLA, and teams can keep their typed agent design discipline while moving production onto a runtime that guarantees it.

Can we add governance on top of PydanticAI with Logfire?

Pydantic AI Harness adds inline guardrails and human approval of tool calls, and Logfire adds tracing and online evaluations. Engineers write those guardrails into each PydanticAI agent, and Pydantic maps them to no named regulation. The EU AI Act expects immutable records witnessed as they happen and classification before a system ships. Akka Verify applies any control to any AI system, guarantees its enforcement, and covers pre-deployment governance.

PydanticAI supports durable execution. Doesn't that cover reliability?

PydanticAI integrates with eight external durable-execution engines, among them Temporal, DBOS, Prefect, Restate, and AWS Lambda, and its own docs frame durability as delegated to those systems. You select, provision, and operate a separate engine and its datastore, and own the SLA, HA/DR, and recovery across the seams. Akka durability is native to the runtime: event-sourced, replayable, sub-1-minute RTO, zero-byte RPO, under a six-nines SLA.

Is PydanticAI cheaper because it is open source?

The framework is open source, but production means the Python infrastructure footprint plus a separate durable-execution engine and the memory, streaming, API, observability, and governance tiers you provision and operate. The Akka shared-compute model is up to 90% cheaper to operate for the same agentic transaction volume, on a fixed annual fee.

Sources

PydanticAI — what it is: ai.pydantic.dev, pydantic.dev/pydantic-ai — model-agnostic, type-safe Python agent framework; structured outputs, dependency injection, MCP, graph workflows

PydanticAI durable execution: pydantic.dev/docs/ai/integrations/durable_execution/overview — eight engines (Temporal, DBOS, Prefect, Restate, AWS Lambda, Absurd, pgtask, Apache Airflow); durability delegated to external engines

PydanticAI maturity: github.com/pydantic/pydantic-ai/releases, pypi.org/project/pydantic-ai — V1 since Sept 2025; V2.0.0 on June 23, 2026; current 2.61.0 (Oct 2026); 1.x security fixes only

Pydantic Logfire: pydantic.dev/logfire, logfire.pydantic.dev/docs/compliance — tracing and evals; SOC 2 Type II, HIPAA, GDPR, EU data region; SLA on Enterprise plan

Pydantic AI Harness & Logfire online evals: pydantic.dev/docs/ai/harness/guardrails, pydantic.dev/articles/online-evals-pydantic-logfire, pydantic.dev/security — inline guardrails with human approval; sampled production scoring; no AI regulation named

Pydantic funding: techcrunch.com (Oct 1, 2024), crunchbase.com — $12.5M Series A led by Sequoia (Oct 2024); ~$17.2M total over two rounds

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 241 regulations and frameworks / 1,718 controls / 699 with financial penalty; 100,000+ deployments / since 2009 / 52 banks; profitable; Dell Technologies Capital